CMPT 120-D200 Midterm Exam
Spring 2026

Duration	50 minutes
Aids allowed	Pencil (or pen) and eraser. No notes, no papers, no books, no computers, no calculators, no cheat sheets, etc.
Scoring	**Multiple Choice Questions** For each question fill in **the one best answer** on the answer sheet. Each correct answer scores 1 point. Incorrect answers, multiple answers, illegible answers, or unanswered questions score 0 points. **Written Questions** Marked using a standard marking scheme.
During the exam	Raise your hand if you would like to speak with a proctor. Questions about exam/course content will **not** be answered during this exam. If there are any changes/corrections to the exam, they will be announced to everyone.

Solutions

1) What number can replace **???** so that this prints 9?

```
print(len('no' + ??? * '!!'))
```

A. 3 B. 4 C. 5 D. there is no number that works

Explanation: Since `'no'` and `'!!'` are both of length 2 (which is even), no amount of them can be concatenated to get a string of length 9 (which is odd).

2) Consider these statements:

i) This statement prints 9.0:

```
print(10 - (20 // 40) * 2)
```

ii) This statement prints 10:

```
print(10 - (20 / 40) * 2)
```

A. i) and ii) are both true
B. i) and ii) are both false
C. i) is true and ii) is false
D. i) is false and ii) is true

Explanation: `//` is integer division, and `/` is float division. In i), `20 // 40` evaluates to 0, and thus the statement is the same as `print(10 - 0 * 2)`, which print 10. In ii), `20 / 40` evaluates to 0.5, and so the statement is the same as `print(10 - 0.5 * 2)`, which prints 9.0.

3) What does this print?

```
print(int(3 / 4))
print(float(3 // 4))
```

A.	B.	C.	D.
`1`	`1`	`0`	`0`
`0.0`	`0.75`	`0.0`	`0.75`

Explanation: `3 / 4` evaluates to 0.75, and then `int(0.75)` evaluates to 0 (`int` converts a `float` to an `int`, chopping off any digits after the decimal). `3 // 4` evaluates to 0 (since `//` is `int` division), and `float(0)` evaluates to `0.0`.

Solutions

4) Which one of these is an **illegal** Python variable name?

A. `s734`
B. `pie_thon_`
C. `Import`
D. `4teen`
E. they are all legal variable names

Explanation: D is the correct answer since a Python variable name cannot start with a number.

5) What does this print?

```
n = 2
n += 2
n *= 2
n /= 4
n -= 1
print(int(n))
```

A. 0 B. 1 C. 2 D. 3 E. 4

Explanation: Tracing the code shows that the answer is 1:

```
n = 2
n += 2          # n is 4
n *= 2          # n is 8
n /= 4          # n is 2
n -= 1          # n is 1
print(int(n))   # 1
```

6) Which program prints the cosine of 10, i.e. cos(10)?

A.
```
import math
print(math.cos(10))
```

B.
```
import math
print(cos(10))
```

C.
```
print(math.cos(10))
```

D. A, B, and C all print the cosine of 10

Explanation: A is the correct way. B is incorrect because it calls `cos` incorrectly (it should be `math.cos`), and C is incorrect because it doesn't import `math`.

Solutions

7) What is the best description of Python source code comments?

A. They are usually ignored by Python and meant as notes to human readers.

B. Python uses them to help speed up the program when it runs.

C. They are attached to functions and can be printed when the program runs.

D. They are notes written by other programmers about issues in the code.

Explanation: A is most accurate.

8) Which one of these code fragments prints the numbers from 1 to 100?

A.
```
for i in range(100):
    print(i)
```

B.
```
for i in range(1, 100):
    print(i)
```

C.
```
for i in range(100):
    print(i + 1)
```

D.
```
for i in range(101):
    print(i)
```

E. none of A, B, C, or D

Explanation: C prints the numbers from 1 to 100. A prints the numbers from 0 to 99, B prints the numbers from 1 to 99, and D prints the numbers from 0 to 100.

9) What does this program print?

```
def f(x):
   a = 2 + x
   print(a)
   a = 6

f(5)
print(a)
```

A.
```
7
6
```

B.
```
7
7
```

C.
```
7
0
```

D. the program has an error

Explanation: The program has an error because the `print(a)` at the end uses the variable `a`, which was defined only in `f` and so cannot be accessed outside of `f`.

Solutions

10) What does this print?

```
print((6 % 10) % (11 % 3))
```

A. 0 B. 1 C. 2 D. 3 E. a number other than 0, 1, 2, or 3

Explanation: `6 % 10` is 6, and `11 % 3` is 2, so the final answer is `6 % 2`, which is 0.

11) What does this print?

```
x = 5
y = 4 + 1
print(x == y)

s = 'pie'
t = 'ball'
print(s <= t)
```

A.	B.	C.	D.
`True`	`False`	`True`	`False`
`True`	`False`	`False`	`True`

Explanation: `x == y` is true because 5 and 4 + 1 evaluate to the same thing. `s <= t` tests if `s` comes before `t` alphabetically (or if they are the same), and so it's `False`.

12) What does this print?

```
a = True
b = False
print(a and not b)
print(a or not b)
```

A.	B.	C.	D.
`False`	`False`	`True`	`True`
`False`	`True`	`False`	`True`

Explanation: `a and not b` simplifies to `True and not False`, which is the same as `True and True`, which is `True`. Similarly, `a or not b` is `True or not False`, which is the same as `True or True`, which simplifies to `True` as well.

Solutions

13) Here's the truth table for Python's `or` operator:

a	b	a or b
True	True	X
True	False	Y
False	True	True
False	False	False

What are the values of X and Y?

A. X is False
   Y is False

B. X is False
   Y is True

C. X is True
   Y is False

D. X is True
   Y is True

Explanation: By definition of "or", the expression `a or b` is `True` when one, or both, of `a` and `b` are `True`.

14) Which programs print 9?

Program 1

```
x = 1
if x > 0:
x = x + 8
print(x)
```

Program 2

```
x = 1
if x > 0:
   x = x + 8
     print(x)
```

A. Both programs print 9
B. Neither program prints 9
C. Program 1 prints 9, and Program 2 doesn't
D. Program 2 prints 9, and Program 1 doesn't

Explanation: Program 1 has no indentation, and so does not run. Program 2 has inconsistent indentation, i.e. an extra space before `print`, and so does not run.

15) What values of `a` and `b` make this code print `cheese`?

```
if not (a or not b):
    print('cheese')
else:
    print('crackers')
```

A. a is False and b is False
B. a is False and b is True
C. a is True and b is False
D. a is True and b is True

Explanation: `'cheese'` is printed just when `not (a or not b)` evaluates to `True`. Since that expression starts with `not`, if it's `True` then `a or not b` must be `False`. That's `False` just when *both* `a` is `False` and `not b` is `False`. So the correct answer is that `a` is `False` and `b` is `True`.

Solutions

16) What values of `x` and `y` make this code print `third`?

```
if x < y:
    print('first')
elif x < y or x == y:
    print('second')
else:
    print('third')
```

A. `x` is 4 and `y` is 5
B. `x` is 4 and `y` is 4
C. `x` is 5 and `y` is 4
D. there are no values of `x` and `y` that make the code print `third`

Explanation: For `'third'` to be printed, both conditions `x < y` and `x < y or x == y` must be `False`. So if `x` is bigger than `y`, `'third'` is printed.

17) What does this program print?

```
def square(n):
    return n * n

def triple(n):
    print(n * 3)

value = square(2) + triple(2)
print(value)
```

A. just 6
B. 6, then and error
C. just 10
D. 10, then an error
E. nothing: an error occurs before anything is printed

Explanation: When the program runs, `square(2)` *returns* 4, and then `triple(2)` *prints* 6. So 6 is printed. But `triple(2)` returns `None`, which is the same as the statement `value = 4 + None`, which causes an error.

Solutions

18) What does this program print?

```
def shout_to(name):
   print(f'Hey {name}!')

x = shout_to('Cassie')
print(x)
```

A. 0

B. nothing: it causes an error

C. `None`

D. nothing: the code runs correctly but it prints nothing

Explanation: The `shout_to` function does not explicitly return a value, and so, by default, it returns the special value `None`.

19) What does this print?

```
n = 0
for i in range(3):
    n += 1
    for j in range(4):
        n += 1
print(n)
```

A. 7 B. 10 C. 12 D. 15

Explanation: Carefully tracing the code, the answer is seen to be 15. Using a table of values for i, j, and n can help.

20) What does this print?

```
n = 0
for i in range(4):
   n -= 1
   for j in range(3):
       n += 1
print(n)
```

A. 3 B. 5 C. 7 D. 9 E. a number other than 3, 5, 7, or 9

Explanation: Carefully tracing the code, it prints 8 (using a table of values for i, j, and n can help). 8 is not one of the given numbers, so E is the correct answer.

Solutions

21) What does this print?

```
def rec(n):
    if n == 0:
        return 0
    else:
        return 2 + rec(n - 1)

print(rec(4))
```

A. 4 B. 7 C. 8 D. nothing: it eventually crashes with an error

Explanation: Tracing the call to `rec(4)` shows the answer is 8:

```
rec(4) = 2 + rec(3)
       = 2 + (2 + rec(2))
       = 2 + (2 + (2 + rec(1)))
       = 2 + (2 + (2 + (2 + rec(0))))
       = 2 + (2 + (2 + (2 + 0)))
       = 8
```

22) What does this print?

```
def f(n):
    if n <= 3:
        return 1
    else:
        return f(n - 1) + f(n - 2) + f(n – 3)

print(f(6))
```

A. 6 B. 7 C. 8 D. 9 E. nothing

Explanation: This is like the Fibonacci sequence, but it starts with 1, 1, 1, and then each following number is the sum of the *three* numbers before it. The sequence is 1, 1, 1, 3, 5, 9, 17 …, and so 9 is the sixth number of the sequence, i.e. the value of `f(6)`.

Solutions

23) What can **???** be replaced with so that `has_lowercase_vowel(s)` works correctly?

```
def has_lowercase_vowel(s):
    """Returns true just when s has one or more
    of the lowercase vowels a, e, i, o, or u. Otherwise it returns False.
    """
    for c in s:
        if ???:
            return True
    return False
```

A. `c in 'aeiou'` B. `c not in 'aeiou'` C. `s in 'aeiou'` D. `s not in 'aeiou'`

Explanation: The for-loop assigns each character in `s` to `c`, and so if any value of `c` is a vowel the function will return `True`. So A is the correct answer.

24) Consider these statements:

```
def has_m(s):
    for c in s:
        if c in 'mM':
            return True
    return False
```

i) If `has_m(s)` returns `True`, then it is **guaranteed** that `s` contains an `'m'`

ii) If `has_m(s)` returns `False`, then it is **possible** that s contain an `'m'`

A. i) and ii) are both true
B. i) and ii) are both false
C. i) is true and ii) is false
D. i) is false and ii) is true

Explanation: i) is false because, for example, `has_m('MMM')` returns `True` but has no `'m'`. ii) is false because if `s` has an `'m'` then the if-statement in the loop will catch it and return `True`.

25) What can **???** be replaced with so that this program prints just the **first line** of the text file `story.txt` ? Assume `story.txt` has multiple lines of text.

```
infile = open('story.txt')
print(???)
```

A. `infile.read()` B. `infile.readline()` C. `infile.read(0)` D. `infile.readline(0)`

Explanation: `infile.readline()` returns the next line of a text file, so the first time you call it it returns the first line of the file. `infile.read()` returns the entire file as a single big string, and `infile.read(0)` and `infile.readline(0)` are syntax errors.

Solutions

26) What does this print? Assume `story.txt` is a text file that is opened without error.

```
infile = open('story.txt')

n = 0
for x in infile:
    n = n + 1
print(n)
```

A. the number of characters in `story.txt`
B. the number of words in `story.txt`
C. the number of lines in `story.txt`
D. something other than the number of characters, lines, or words in `story.txt`

Explanation: The for-loop goes through each line of the file, assign the line to `x`. So the final value of `n` is the number of lines in the file.

27) If `s` is a string, then `s.strip()` returns a copy `s` with:

A. all whitespace characters removed
B. just leading whitespace characters removed
C. just trailing whitespace characters removed
D. just leading and trailing whitespace characters removed

Explanation: `strip()` returns a copy of the string with whitespace as the beginning and end removed.

28) What can **???** be replaced by so this code prints **9** ?

```
t = 0
for i in range(3):
    for j in range(???):
        t += i + j

print(t) # prints 9
```

A. 1 B. 2 C. 3 D. 4

Explanation: By tracing the code start with **???** replaced by 1 will lead to seeing that B is correct.

Solutions

29) How many * characters does this print?

```
a = 1
b = 10
while a < b:
    print('*')
    a += 1
    b -= 1
```

A. 3 B. 4 C. 5 D. 6 E. it's an infinite loop, so it prints an infinite number of * characters

Explanation: Tracing the code shows that 5 * characters are printed:

a	b	What's printed
1	10	*
2	9	*
3	8	*
4	7	*
5	6	*

Inside the loop as long as `a` is less than `b` a * is printed. We can see that in total 5 *s are printed.

30) What does this print?

```
t = 0
for i in range(5):
    j = 0
    while j < i:
        t += j
        j += 1

print(t)
```

A. 10 B. 15 C. 20 D. 35

Explanation: Tracing this code shows that the final value of `t` is 10.

Solutions

PIN Codes

a) (3 marks) Write a function called `is_digit(s)` that returns `True` if the string `s` is a single digit, and `False` otherwise. The digits are `'0'` through `'9'`, and assume `s` is a string.

For example:

```
print(is_digit('5'))   # True
print(is_digit('52'))  # False
print(is_digit('2w0')) # False
```

In addition:

- Do not use the built-in `s.isdigit()` function in any way.
- Do not import anything.
- Do not use lists or tuples. Stick to features of Python discussed so far in the course.
- Make your function efficient, readable, and short.
- We want to see **perfect** syntax.

Sample Solution

```
def is_digit(s):
    return len(s) == 1 and '0' <= s <= '9'
    # return len(s) == 1 and s in '0123456789'
    # return s == '0' or … or s == '9')
```

Solutions

b) (7 marks) Strings `s` and `t` are called a **good pair** just when the following things are true:

- `s` and `t` are both non-empty strings of digits (no non-digit characters).
- `s` and `t` don't have any digits in common.

Write a **boolean function** called `good_pair(s, t)` that returns `True` if `s` and `t` are a good pair, and `False` otherwise. Use `is_digit` from the previous question to help (you don't need to re-write it), and assume `s` and `t` are both strings.

For example:

```
print(good_pair('477', '08125'))  # True
print(good_pair('91234', '56'))   # True
print(good_pair('477', '5048'))   # False
print(good_pair('4f7', '08125'))  # False
print(good_pair('477', ''))       # False
```

In addition:

- Do not import anything.
- Do not use lists or tuples. Stick to features of Python discussed so far in the course.
- Make your function efficient, readable, and short.
- We want to see **perfect** syntax.

```
def good_pair(s, t):
    if len(s) == 0 or len(t) == 0: # check if s or t empty
        return False
    for c in s + t:                # check s and t are all digits
        if not is_digit(c):
            return False
    for c in s:                    # make sure no characters in common
        if c in t:
            return False

    return True
```

Solutions